

# LESLIE KADENGE

Game Developer

+254 794 383225 · [lesliekadenge@gmail.com](mailto:lesliekadenge@gmail.com) · Nairobi, Kenya · [github.com/les-k](https://github.com/les-k) · [linkedin.com/in/leslie-kadenge-4a4b62213](https://linkedin.com/in/leslie-kadenge-4a4b62213)

## SUMMARY

Information Technology graduate whose interest is the half of game development almost nobody writes about: testing the rules. Game logic feels verified by playing it, and it is not — the bugs that matter are invisible while you play.

The work below separates rules from rendering so the rules can be tested without a browser, and seeds every source of randomness so any session can be replayed exactly.

## PROJECTS

### 2048 — the rules, tested JavaScript · node:test · zero dependencies

[github.com/les-k/2048](https://github.com/les-k/2048)

- 15 tests on Node's built-in runner, no dependencies and no build step, running on Node 20, 22 and 24.
- Three defects the suite exists to catch, none of which throws an error or looks wrong in a screenshot: a tile merging twice in one move, merging resolved from the wrong end, and a tile spawning after a move that changed nothing — the last quietly making a game unwinnable.
- Tiles are placed by a seeded mulberry32 generator passed in as an argument rather than Math.random, so the same seed and moves reproduce a board exactly on any machine.
- Rules live in pure functions over a flat array with no DOM, no clock and no randomness reached for internally, so a failing case is written down as two literal grids and CI needs no browser.
- One defect the suite could not catch, found by someone playing it: a class setting display outranked the browser's own [hidden] rule, so the overlay covered the board while element.hidden still reported true.

## APPROACH

Seeded randomness is what makes a game testable at all. Math.random cannot be seeded, so a game that uses it can never be replayed and a failing test can never be reproduced. Passing the generator in as an argument, rather than reaching for it, is the decision everything else depends on.

The same separation makes property-based testing possible: rather than asserting one board behaves correctly, generate a thousand seeded ones and assert a property holds across all of them.

## TECHNICAL SKILLS

**LANGUAGES** JavaScript, Python, C++ (coursework)

**GAME** Game loops, state machines, seeded RNG, deterministic replay, grid logic, Unity

**TESTING** node:test, property-based testing, regression suites, CI across runtimes

**FRONTEND** Canvas and DOM rendering, touch input, accessible game interfaces

**TOOLING** Git, GitHub Actions, GitHub Pages

## EDUCATION

**BSc Information Technology** — Zetech University, November 2023 Second-class honours, upper division

Coursework: Machine Learning, Data Science, Artificial Intelligence, Database Systems, Software Engineering, Distributed Systems, Programming Fundamentals (Python, Java, C, C++, JavaScript).

## CERTIFICATION

ICDL Profile — Computer Pride.

*References available on request.*